NEURALSENTINEL

Documentation

vicomtech

MEMBER OF BASQUE RESEARCH
& TECHNOLOGY ALLIANCE

Contents

1. INTRODUCTION 1
2. BACKGROUND 2
2.1 JUSTIFICATION 2
2.2 MOTIVATION 2
3. STATE OF THE ART 4
3.1 ADVERSARIAL ROBUSTNESS EVALUATION AND MODEL SECURITY TOOLS 4
3.2 DATASET AUDITING AND FAIRNESS TOOLKITS 5
3.3 COMPLIANCE-FOCUSED AI GOVERNANCE PLATFORMS 6
3.4 PRIVACY-PRESERVING AND DATA SECURITY TOOLS 8
3.5 NOTABLE STRENGTHS AND GAPS IN THE CURRENT LANDSCAPE 9
4. DEVELOPMENT 11
4.1 MODEL AND DATASET FORMATS JUSTIFICATION 11
4.2 SECURITY IN THE APP 12
4.3 DETAIL OF EACH SCREEN 14
4.4 CONTINUOUS SUPERVISION MODULE 22
4.5 IMPLEMENTATION CONSIDERATIONS 23
DEFINITIONS, ACRONYMS, ABBREVIATIONS 31
APPENDICES 33

1. INTRODUCTION

As artificial intelligence (AI) becomes a cornerstone for innovation in numerous sectors, ranging from finance to healthcare, the importance of systematic security evaluations for AI systems grows increasingly significant. Ensuring that models are free from vulnerabilities and that datasets adhere to privacy and ethical standards is not merely a best practice: it is rapidly becoming a critical step in certification processes. The AI Act and related regulations underline the pressing need to quantify model behaviors through explicit metrics, thereby enabling consistent and transparent risk assessments.

This project, titled NeuralSentinel, is conceived to address precisely these challenges. Through a carefully designed interface, end users can seamlessly upload their datasets and pretrained AI models. The system will then analyze and return a set of core security metrics aligned with recognized standards. While the project initially focuses on essential metrics, it also provides an on- demand mechanism for extended functionalities through dedicated plugins. This plugin-based approach ensures that specialized or emerging metrics, or even custom data visualizations or transformations, can be added to the application without restructuring the main software significantly.

Moreover, the project includes a dedicated component that monitors the usage environment, leveraging machine learning to track and validate user behavior. This feature not only enhances data privacy and model protection but also fortifies the continuous authentication strategy, mitigating potential insider threats and unauthorized access attempts.

In essence, the project aspires to merge robust security assessment practices with a stable architecture, preparing for future expansions in scope and complexity.

2. BACKGROUND

2.1 JUSTIFICATION

The widespread adoption of machine learning (ML) and deep learning (DL) systems has unveiled unprecedented risks across the entire model lifecycle, from training to inference. For example, adversarial attack techniques can compromise model robustness in production, resulting in erratic or deliberately manipulated predictions. Moreover, attack vectors such as backdoors during pre-training or data poisoning have been systematically catalogued in recent studies, showing that no stage is immune to vulnerabilities.

At the same time, data privacy has become a critical challenge. In Big Data and collaborative-modeling contexts, risks of sensitive information leakage and inference attacks against training data are growing. Recent surveys of privacy-preserving machine learning emphasize the need for techniques, such as secure multi-party computation (MPC), homomorphic encryption, and differential privacy, to guarantee confidentiality without sacrificing model utility.

On the regulatory front, the OECD AI Principles (2019) promote AI that is “innovative and trustworthy and that respects human rights and democratic values”, and the OECD Council Recommendation on AI (2024) reaffirms these principles, insisting on “robustness, security and safety” as core governance pillars. Similarly, the EU AI Act reinforces the obligation to measure and report explicit security and privacy metrics as part of model certification. Finally, the NIST AI Risk Management Framework (AI RMF) makes it clear that trustworthy AI requires systematic risk measurement throughout design, development, use, and evaluation.

In sum, today’s landscape combines:

1. New threat vectors throughout the ML lifecycle
2. Privacy challenges demanding PPML and anonymization techniques
3. Global regulatory pressures (OECD, AI Act, NIST AI RMF), requiring explicit metrics and transparent auditing

Together, these converging factors underscore the necessity of a unified platform that integrates security metrics, privacy protections and compliance checks, enabling organizations to deploy AI systems with confidence.

2.2 MOTIVATION

From an economic standpoint, organizations face potential legal and reputational risks if their AI systems are compromised or found to be non-compliant. According to IBM’s 2024 Cost of a Data Breach Report, the global average cost of a data breach reached USD 4.88 million in 2024, a 10 % increase over the previous year and the highest total ever. Moreover, the same study shows that enterprises extensively using security AI and automation save an average of USD 2.22 million in avoided breach costs, underscoring the ROI of proactive security measures.

Faced with regulatory measures such as the EU AI Act, NIS2, DORA, and GDPR, businesses urgently require tools that quantify compliance and reduce administrative overhead. A solution like NeuralSentinel, with an intuitive interface and automatic calculation of metrics, represents a strategic asset with strong commercial potential across finance, healthcare, mobility, and beyond.

On the technical side, a plugin-based architecture delivers native extensibility: new metrics or data-transformation modules can be added without touching the core, accelerating adaptation to emerging requirements. Together, these design choices ensure a scalable, updatable system aligned with modern software-engineering best practices.

Finally, integrating ML into the supervision module not only strengthens continuous authentication but also enables real-time anomaly detection, closing the defensive loop with active monitoring and automated response to insider threats. This vigilance is essential for any high-stakes AI deployment.

3. STATE OF THE ART

Modern AI systems face a range of security and trustworthiness challenges: from adversarial attacks and data poisoning to issues of fairness, privacy, and regulatory compliance. In developing NeuralSentinel, it is crucial to survey existing tools and platforms addressing these concerns. The state-of-the-art can be grouped into several categories:

1. Adversarial robustness evaluation frameworks
2. Dataset auditing and bias/fairness toolkits
3. Compliance-focused AI governance platforms, and
4. Privacy-preserving ML tools.

For each category, representative solutions will be reviewed, highlighting their key functionalities, their alignment with NeuralSentinel's approach where appropriate, and notable strengths or gaps.

3.1 ADVERSARIAL ROBUSTNESS EVALUATION AND MODEL SECURITY TOOLS

One major class of tools focuses on adversarial robustness, evaluating and improving a model's resilience to attacks. A prime example is IBM's Adversarial Robustness Toolbox (ART), an open-source library introduced in 2018 to "evaluate the security of machine learning models through various means". ART has since become a Linux Foundation project, providing a comprehensive suite of 39+ attack methods and 29+ defense techniques covering evasion attacks (adversarial examples), data poisoning, model extraction, and inference (membership) attacks. It supports numerous data types (images, text, audio, etc.) and ML frameworks (TensorFlow, PyTorch, scikit-learn, XGBoost, etc.), allowing users to create, attack, and evaluate models all in one toolkit. Built with extensibility in mind, ART includes modular metrics for robustness and even certification/verification measures. This breadth aligns with NeuralSentinel's goal of handling commonly used formats and providing security metrics. However, ART is primarily a Python library, and it lacks a dedicated user interface or interactive dashboard. NeuralSentinel can draw on ART's rich attack/defense algorithms while aiming to offer a more user-friendly front-end and use the plugin architecture for integration.

Another notable tool is Microsoft Counterfit, released in 2021 as an automation platform for AI red-teaming. Counterfit is a command-line interface (CLI) toolkit that acts as a generic layer to orchestrate attacks on ML models. Rather than re-implementing attacks, Counterfit integrates with existing adversarial toolkits (including IBM ART, TextAttack for NLP, etc.), allowing security professionals to easily run penetration tests on models in a uniform environment. It is "environment-agnostic and model-agnostic", capable of targeting models on any cloud or on- premise and accepting various input data types (text, images, audio). Microsoft designed Counterfit to help organizations perform AI security risk assessments, ensuring their models are "robust, reliable, and trustworthy". In practice, some sources claim Counterfit's approach is analogous to a simplified Metasploit-style framework for AI, where users can script attack scenarios and generate reports. This complements NeuralSentinel's vision by emphasizing

automation and extensibility (Counterfit even allows users to contribute custom attack scripts). However, Counterfit is CLI-based and geared to security experts.

Academic and industry research groups have also contributed adversarial robustness tools. CleverHans (by the CleverHans Lab) was one of the earliest libraries for adversarial attacks/defenses, providing reference implementations of attacks like FGSM, Carlini-Wagner, and others for benchmarking across TensorFlow, PyTorch, etc. It constitutes a code-focused tool, mainly for researchers to test new attack methods, and is less full-featured than ART. SecML is another library that emphasizes security, offering a framework to test evasion and poisoning attacks on various classifiers. Meanwhile, the U.S. DARPA's GARD program released Armory, a "containerized testbed for adversarial robustness evaluation" (though some argue it is more research-oriented). Armory uses scenario configuration files to launch evaluations and heavily leverages ART under the hood for attack implementations. The value of Armory lies in its ability to conduct scalable, repeatable experiments, for example, testing a defense technique against a suite of attacks in a controlled, isolated environment. Armory thus targets a similar need as NeuralSentinel (systematic evaluation of model defenses) but is oriented towards researchers validating defense effectiveness. NeuralSentinel can take inspiration from Armory's modular scenario approach while aiming for a more accessible, GUI-driven, workflow for end-users.

In summary, adversarial robustness tools provide critical capabilities for evaluating model vulnerability and resilience. They show an emerging trend towards integration and extensibility: e.g. ART's modular design and Counterfit's unification of frameworks. A noted gap is that few current tools offer end-to-end pipeline integration.

3.2 DATASET AUDITING AND FAIRNESS TOOLKITS

Beyond model attacks, the integrity and fairness of training data is another aspect of AI security. Biases or quality issues in datasets can lead to unsafe or unethical AI outcomes. A number of toolkits have been developed to audit datasets and model outputs for fairness, bias, and other ethical risks. IBM's AI Fairness 360 (AIF360) is a library that provides "a comprehensive set of metrics for datasets and models to test for biases" and includes algorithms to mitigate bias. It contains statistical fairness metrics (e.g. disparate impact, equal opportunity difference) to detect bias across protected attributes, along with explainers for those metrics and preprocessing or postprocessing techniques to reduce identified biases. By spanning the AI lifecycle (data preprocessing, in-training adjustments, and post-hoc corrections), AIF360 is well-aligned with AI Act principles of non- discrimination and transparency. NeuralSentinel, which plans to offer metrics aligned with the AI Act, can leverage such fairness metrics to ensure datasets are "secure" in the sense of not embedding harmful biases. However, AIF360 is code-centric (Python/R library) and primarily aimed at data scientists. NeuralSentinel could provide a more interactive auditing interface, guiding even non- experts to detect bias in their datasets or model decisions (perhaps via plugins that wrap AIF360's metrics).

Another relevant tool is Aequitas, an open-source bias audit toolkit from the University of Chicago's Center for Data Science and Public Policy. Aequitas provides an "easy-to-use and transparent tool for

auditing predictors of ML models" (which, however, requires statistical knowledge), with a focus on group fairness in binary classification. It produces reports on metrics like false positive/negative rates across demographic groups and can suggest "bias mitigation" strategies by adjusting decision thresholds. Tools like Aequitas and Microsoft Fairlearn (a fairness assessment and mitigation library from Microsoft Research) share similar goals: to empower developers to assess and improve fairness of AI systems. They typically include visualization dashboards (e.g., Fairlearn's dashboard) to compare model performance for different subgroups, which is a feature NeuralSentinel could emulate for user-friendly interaction. The strength of these toolkits is in domain-specific metrics and visualizations for ethical AI concerns, and a gap is that they usually operate offline or in notebooks rather than being integrated into evaluation platforms.

Microsoft Fairlearn's dashboard widget [1]

Beyond bias, dataset auditing extends to data quality and integrity, essential for security as corrupted or low-quality data can compromise model behavior. While not explicitly "security" tools, frameworks like Great Expectations or TensorFlow Data Validation help profile and validate datasets (detecting anomalies, missing values, schema mismatches). These ensure that training data hasn't been unexpectedly altered, which could also catch certain data poisoning attempts. Some research has also looked at detecting backdoor or Trojan triggers in datasets/models (e.g., the Neural Cleanse technique scans for anomalous patterns that could indicate a Trojan backdoor). Such methods are largely research prototypes and not yet in widespread toolkit form, representing a gap in current state-of-the-art tooling.

3.3 COMPLIANCE-FOCUSED AI GOVERNANCE PLATFORMS

As AI regulations mature (e.g., the now operative EU AI Act), a new category of platforms is emerging to help organizations ensure their AI systems comply with legal and ethical standards. These AI governance and compliance platforms go beyond technical testing to provide workflow tools for risk management, documentation, and oversight of AI throughout its lifecycle. A representative example is Enzai, which offers an AI governance tool aimed at managing AI risks while "meeting emerging regulatory obligations". Enzai's platform is built with the EU AI Act in mind: it includes an "AI Act

[1] Source: https://fairlearn.org/v0.5.0/quickstart.html

Compliance Framework" that guides users through the required steps (from initial risk classification of an AI system to a full conformity assessment). Essentially, Enzai translates the text of the EU Act into a series of questions and checklist items, ensuring nothing gets overlooked in demonstrating compliance. Notably, Enzai can continuously measure key compliance metrics such as AUC, bias, and fairness via a comprehensive API. It also collects required documentation (technical documentation, instructions for use, etc.) in a centralized way. This approach aligns with NeuralSentinel's aim to report metrics consistent with the AI Act: for instance, tracking a model's accuracy and robustness (for technical compliance), and fairness metrics (for societal impact), all of which the Act expects high-risk AI systems to document. The strength of Enzai and similar platforms is the end-to-end governance workflow: they integrate technical metrics with policy checklists and reporting tools. However, these are often commercial SaaS platforms (Enzai, for example, is a proprietary tool), and they may not expose low-level security testing capabilities (like adversarial attack simulations) directly to users.

Another notable player is Credo AI, an enterprise platform for "Governance, Risk, and Compliance (GRC)" for AI. Credo AI emphasizes continuous oversight through workflows that ensure AI systems align with internal policies and external regulations throughout their lifecycle. It provides features like an AI registry (inventory of models), risk assessment questionnaires, bias and performance analytics, and policy mapping. For example, Credo's platform can map metrics from a model (say, its fairness or robustness scores) to requirements of the AI Act or NIST AI Risk Management Framework. As a "GRC for AI" solution, Credo AI underscores that ensuring AI trustworthiness is not a one-time checkbox but requires ongoing monitoring and management of risks.

Also relevant are the model monitoring services provided by major cloud providers, often framed as responsible AI or trust monitoring. IBM Watson OpenScale, for instance, is a production AI monitoring tool that tracks deployed models for fairness, drift, and quality issues, and can even explain model decisions in real-time. It allows organizations to set thresholds (e.g., a maximum allowable bias or performance drop) and will trigger alerts or even auto-mitigations if they are exceeded. This kind of live monitoring ensures that models remain within the safe bounds defined by compliance standards and that any degradation or anomalous behavior is caught early. Microsoft Azure and Google Cloud offer similar capabilities (Azure's Responsible AI dashboard, GCP's model monitoring for drift and outlier detection, etc.), though these are typically tied to their ecosystems. NeuralSentinel's design, being MVC-based and extensible, could integrate with such monitoring to provide an independent, cross-platform oversight tool.

By incorporating ML-based authentication and anomaly detection in its continuous supervision module, NeuralSentinel could not only watch the model's outputs but also guard the system itself, detecting unusual user activity. This is an area not explicitly covered by current AI governance platforms, which focus more on model outputs and documentation rather than securing the user access layer. Thus, NeuralSentinel has an opportunity to bridge security monitoring with compliance monitoring in one extensible application.

In summary, compliance-focused AI platforms underscore that security and ethics go hand-in-hand under regulatory scrutiny. They provide high-level frameworks to ensure AI is used responsibly

(meeting criteria of fairness, transparency, accuracy, etc.), and they increasingly incorporate technical metrics as evidence. NeuralSentinel, aiming to provide metrics aligned with the AI Act, could benefit from the metrics defined by these platforms (e.g. tracking accuracy, robustness, bias as key indicators and can fill gaps by offering deeper technical evaluations (adversarial robustness tests, data audits) within a governance context. One gap to note is that many governance tools are still evolving to handle more AI security-specific metrics (cybersecurity): the AI Act explicitly calls for robustness and cybersecurity testing for high-risk AI, so a tool like NeuralSentinel that specializes in security evaluation could augment compliance efforts significantly.

3.4 PRIVACY-PRESERVING AND DATA SECURITY TOOLS

Protecting the privacy and confidentiality of data used in AI is another critical aspect of AI security. Tools in this category help ensure that models do not leak sensitive information and that datasets can be utilized without compromising individual privacy. A key technique is differential privacy (DP), which involves adding statistical noise to data or training processes to give formal privacy guarantees. There are several libraries making DP accessible. The IBM Differential Privacy Library (Diffprivlib), for example, offers a “suite of tools for machine learning and data analytics tasks, all with built-in privacy guarantees”. It provides DP-enabled versions of common algorithms (like logistic regression, clustering, PCA) and a “privacy budget” accountant to track how cumulative operations spend this budget. This allows developers to experiment with DP by changing just a line or two of code, and obtain models that provably limit the influence of any single data point on the output. Google’s TensorFlow Privacy and Meta’s PyTorch Opacus similarly enable training neural networks with differential privacy by altering the training procedure (gradient clipping and noise addition). These tools are invaluable for scenarios where datasets contain personal data, as using them can help a system comply with regulations like GDPR by minimizing the risk of re- identification from model outputs.

Another facet of privacy in AI is membership inference and privacy auditing. Researchers have developed methods to test whether a trained model inadvertently exposes data from its training set (a form of information leakage). The ML Privacy Meter is an open-source audit tool designed for this purpose. It “enables data protection impact assessment based on state-of-the-art membership inference attacks”: effectively, it tries to infer if specific data points were part of the model’s training data. By simulating known attacks, Privacy Meter quantifies a model’s privacy risk score (although it is worth mentioning that this metric is not standardized). For example, a high success rate of membership inference indicates the model memorizes data too much and could violate privacy of individuals in the dataset. Such a tool is highly relevant to AI Act compliance, as the Act (along with laws like GDPR) emphasizes data protection and requires assessing risks to personal data.

In the domain of privacy-preserving machine learning, other tools include federated learning frameworks (like TensorFlow Federated, OpenFL by Intel, or PySyft by OpenMined) which allow training on distributed data without centralizing raw datasets. These address security of data by design (data never leaves its source), though they come with their own evaluation tools for convergence and potential attack vectors (e.g., federated models can still be attacked or poisoned by malicious participants). Homomorphic encryption libraries and secure enclaves (e.g., Microsoft SEAL

for encrypted inference, or Nvidia's confidential AI infrastructure) are also noteworthy, but they tend to be lower-level, and not as directly comparable to NeuralSentinel's scope (which is evaluating models/datasets post-hoc, rather than providing new training protocols).

The strength of privacy tools lies in offering measurable privacy guarantees or risk metrics. For instance, differential privacy gives an ε-value that quantifies the privacy loss, and membership inference tools give empirical metrics of leakage.

A gap in current practice is that using such privacy techniques often require specialized knowledge and significant trade-offs (e.g., DP can reduce model accuracy, and privacy audits require careful experiment setup).

3.5 NOTABLE STRENGTHS AND GAPS IN THE CURRENT LANDSCAPE

Across all these categories, we observe that the field is moving towards integrated, extensible solutions for trustworthy AI. Academic prototypes (like ART, AIF360, CleverHans, Privacy Meter) typically excel in depth for a specific area (robustness, fairness, etc.) and are often open-source, allowing the community to contribute new techniques. Commercial platforms (like Enzai, Credo AI, Holistic AI's governance suite, IBM OpenScale) excel in breadth and usability, combining multiple aspects (metrics, monitoring, documentation) into unified workflows aimed at compliance and enterprise adoption.

For NeuralSentinel, which aspires to evaluate dataset/model security in a holistic yet modular way, the strengths to emulate include: the comprehensive attack/defense libraries of adversarial toolkits, the rich metric sets and visualizations of fairness and privacy tools, and the user-centric design of governance platforms. Its novel opportunity is to blend these strengths into one application. The MVC-inspired architecture and plugin design indicate that NeuralSentinel can incorporate best-of- breed algorithms from existing libraries as "plugins" (e.g., one plugin for adversarial attack evaluation, one for bias metrics, one for privacy checks), something few current tools do across domains.

In terms of gaps to address, a few stand out. Many tools operate in silos (focusing on one dimension of security or one stage of the AI lifecycle). This can lead to fragmented assessments: for example, one might use ART for adversarial testing, and separately use AIF360 for bias checking, but manually combining those insights is difficult. NeuralSentinel can fill this gap by providing a centralized evaluation covering multiple security dimensions side by side. Another gap is ongoing evaluation. Real-world AI deployments need ongoing surveillance because new threats or drifts can emerge over time. Outside of production services like OpenScale or Azure Monitor (which are proprietary), most academic tools perform one-time analysis.

Finally, aligning with the AI Act metrics means NeuralSentinel will likely report on transparency, accuracy, robustness, and possibly "cybersecurity" of the AI system. While accuracy is straightforward, robustness and cybersecurity in AI contexts are still loosely defined in practice. The surveyed tools contribute building blocks: robustness can be measured via adversarial attack success rates (from ART/Counterfit), and cybersecurity might encompass resistance to model extraction or

data leakage (from Privacy Meter or similar). By leveraging these, NeuralSentinel could expand to pioneer a more concrete set of security metrics for AI compliance, helping to operationalize what the AI Act expects (e.g., stating that a model has X% accuracy on a test set, withstands Y% of simulated adversarial attacks up to a certain perturbation, has no severe bias according to certain indices, and leaks below Z% of private information under attack). No existing platform yet provides all that information in one package, and this is a gap NeuralSentinel could position itself to fill in the future.

In conclusion, the state-of-the-art provides a rich toolkit landscape addressing adversarial robustness, data audits, responsible AI compliance, and privacy. NeuralSentinel can be viewed as an ambitious effort to unify these threads: much like a “sentinel” overseeing multiple facets of AI security. By studying and integrating the strengths of prior tools (while minding their limitations), NeuralSentinel’s development can push the frontier of AI security evaluation, offering a uniquely comprehensive and extensible platform aligned with emerging regulatory and ethical requirements in the AI domain.

4. DEVELOPMENT

4.1 MODEL AND DATASET FORMATS JUSTIFICATION

In the development of NeuralSentinel, selecting appropriate formats for both datasets and models is crucial to ensure compatibility, efficiency, and scalability. This section provides an analysis of commonly used formats in the machine learning community.

4.1.1 Dataset formats

Efficient data storage and retrieval are vital for machine learning applications. Several formats are prevalent:

- **CSV (.csv):** A text-based format that is human-readable and widely supported. However, it lacks support for complex data types and can be inefficient for large datasets due to its size and parsing overhead.
- **NumPy (.npy):** A binary format designed for storing NumPy arrays. It offers efficient storage and quick loading times, making it suitable for large datasets. NumPy arrays are the standard for numerical data in Python, ensuring compatibility with various libraries.
- **HDF5 (.h5):** A hierarchical data format that supports large, complex datasets. It allows for compression and efficient slicing, which is beneficial for handling subsets of data without loading the entire dataset into memory.
- **NPZ (.npz):** A compressed archive of multiple NumPy arrays in a single file, ideal for bundling feature (x) and label (y) arrays together. It combines the speed of .npy with on-the-fly compression, reducing disk usage while preserving array structure and metadata.

Given these options, the .npy format is particularly advantageous for single-array datasets in NeuralSentinel due to its simplicity, speed, and seamless integration with NumPy-based workflows, while .npz files offer a convenient, space-efficient way to package paired arrays (e.g. x and y) in one archive.

4.1.2 Model formats

Storing trained machine learning models requires formats that preserve both the architecture and learned parameters. Common formats include:

- **Pickle (.pkl):** Python's native serialization format. While flexible, it poses security risks and is not framework-agnostic.
- **ONNX (.onnx):** An open format designed for interoperability between different machine learning frameworks. It facilitates model sharing but may not support all features of specific frameworks.
- **HDF5 (.h5):** A versatile format that can store the complete state of a model, including architecture, weights, and optimizer configurations. It is the default format for saving models in Keras and is supported by various other frameworks.

The .h5 format is particularly suitable for NeuralSentinel as it encapsulates all necessary components

of a model in a single file, facilitating easy storage, transfer, and loading across different environments.

Given that and the NeuralStrength Lite's requirement of Python 3.9, the following versions are supported:

- **TensorFlow 2.10.0**, as it is the last release that officially supports Python 3.9 without requiring development or pre-release builds, ensuring true "plug & play" integration in embedded environments like this one.
- **NumPy 1.25.0**, released in April 2025, which offers significant performance optimizations and bug fixes while remaining fully compatible with TensorFlow builds of that generation.

4.2 SECURITY IN THE APP

Security is a top priority in the development of NeuralSentinel. The application has been designed with a robust set of security measures to protect users and their data, following the best practices outlined in the official Electron Security Guidelines. These measures are put in place to ensure that the app is secure now and in the future. These include:

1. Secure content loading

All content, including renderer pages and assets, is served securely using our custom app://-/ protocol or local file:// URLs. This eliminates any security risks associated with insecure HTTP connections or mixed content, protecting the app from potential man-in-the-middle attacks.

2. Node.js integration disabled

Node.js integration is disabled to prevent renderer scripts from accessing Node.js APIs. This protects the app from the risk of remote code execution, ensuring that any untrusted content or scripts cannot harm the system.

3. Context isolation enabled

With context isolation enabled, the application ensures that the core application code and renderer scripts operate in separate JavaScript contexts. This prevents unauthorized manipulation of core objects, reducing the risk of tampering with sensitive data.

4. Process sandboxing

By leveraging Chromium's OS-level sandboxing, the app restricts access to sensitive resources like the file system. This significantly limits the potential attack surface and provides an added layer of defense against malicious behavior.

5. Permission request handling

Permission requests are only allowed from trusted sources. The app denies any unauthorized requests for sensitive permissions, such as notifications, to prevent unapproved access.

6. Web security remains enabled

The app keeps web security features enabled, including the same-origin policy, which protects it from cross-origin data leaks and script injections. This ensures that external content cannot interfere with the app's operations.

7. Content security policy (CSP)

A strict CSP is defined to block inline scripts and unauthorized external hosts, reducing the risk of cross-site scripting (XSS) attacks. This adds an additional layer of protection to the app's

content. Disallow insecure content

The app blocks mixed content, meaning HTTPS pages cannot load insecure HTTP resources. This ensures that all external data is transferred securely.

8. Experimental features disabled

   We avoid enabling experimental Blink features, ensuring that only stable, tested behaviors are present in the app, reducing the risk of introducing unassessed vulnerabilities.

9. No <webview allowpopups> usage

   All <webview> elements are carefully analyzed before being added to the app. Unsafe attributes are removed, and only trusted origins are allowed, preventing the app from loading potentially harmful content.

10. <webview> options verification

    When adding a <webview>, its webPreferences are sanitized, disabling preload injections and Node.js integration. Only approved sources are allowed to load, reducing the chances of unauthorized code execution.

11. Navigation control

    The app strictly controls navigation within the app. Handlers are in place to prevent users from navigating away from trusted content, preventing phishing attempts through redirection.

12. Window creation control

    New windows are only allowed for external links from trusted sources. Any other pop-ups are blocked, ensuring that unwanted or potentially harmful windows cannot be opened.

13. Controlled shell.openExternal

    Links are only opened in the user's default browser if they point to our trusted domain. This prevents links from untrusted sources, ensuring the app only directs users to safe websites.

14. Current Electron version

    The app uses the latest version of Electron (v25), which ensures that all security patches from Chromium and Node.js are included, reducing the risk posed by known vulnerabilities.

15. IPC sender validation

    All Inter-Process Communication (IPC) handlers validate the origin of the sender's request. Any calls from untrusted sources are rejected, ensuring that only authorized components can trigger sensitive actions within the app.

16. Custom protocol over file://

    The app uses a custom app://-/ protocol instead of file://, which provides better control over URL resolution and prevents unauthorized access to files outside the app's scope.

17. Fuse configuration audited and locked down

    Unnecessary fuses, such as runAsNode, are disabled during the build process to prevent debugging or injection via environment variables or command-line flags.

18. Minimal API exposure

    The app does not expose raw Electron or IPC APIs directly to renderer scripts. Instead, it uses a secure preload bridge that only exposes explicitly allowed methods, ensuring that external scripts cannot access sensitive application features.

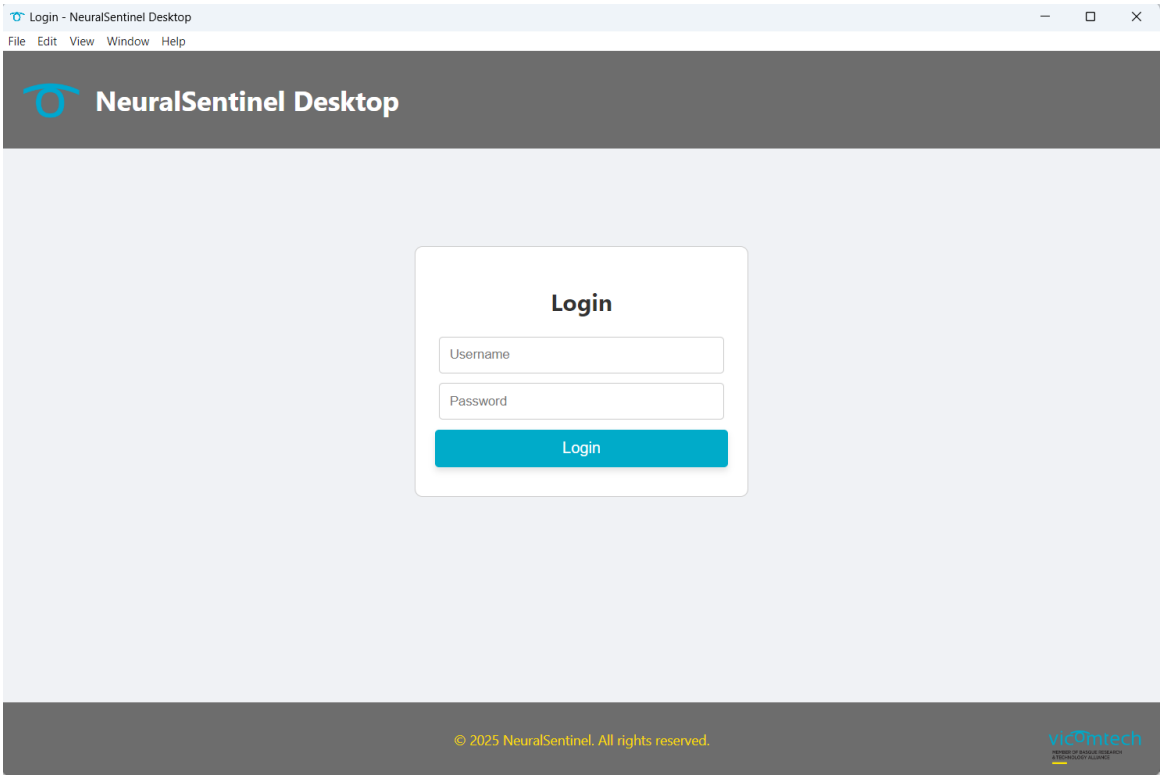
By following these security measures, NeuralSentinel Desktop has a robust security configuration that minimizes risks and ensures users have a safe and trustworthy experience. These practices are

not only useful now: they will continue to apply and evolve as new features are added to the app, maintaining a strong security foundation throughout the development lifecycle, adequate for the extensibility of this app.

4.3 DETAIL OF EACH SCREEN

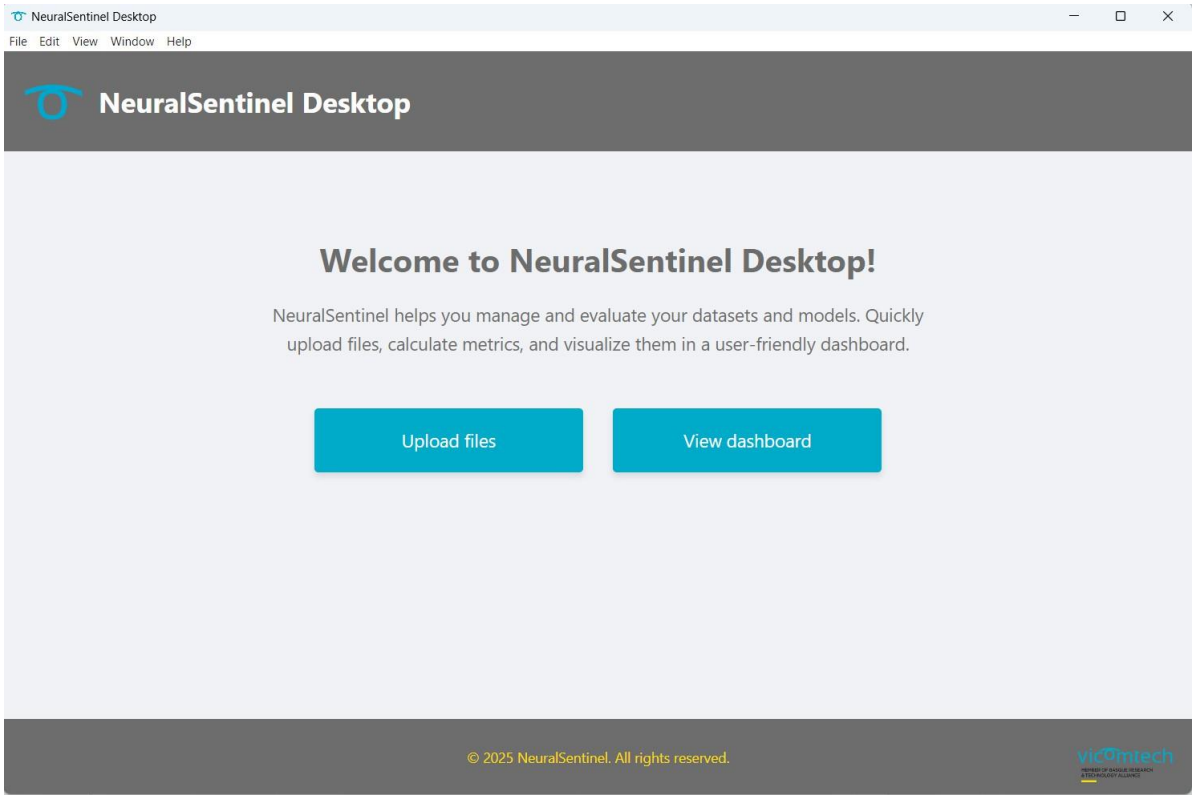
4.3.1 Login screen

The login screen is the gateway to NeuralSentinel Desktop, where users authenticate before accessing the main features of the application. It ensures that only authorized users can interact with the system, safeguarding data and processes. Upon successful login, the user is granted access, and background processes such as the data collector and continuous authenticator are automatically started. The data collector gathers user behavior data, while the authenticator continuously monitors activity to detect anomalies, enhancing security. The design is simple and user-friendly, with clearly labeled fields for username and password. In case of invalid credentials, an error message is displayed, guiding the user to correct the login details. The screen also maintains consistent branding with the banner at the top and the Vicomtech logo at the bottom, reinforcing the application's identity.



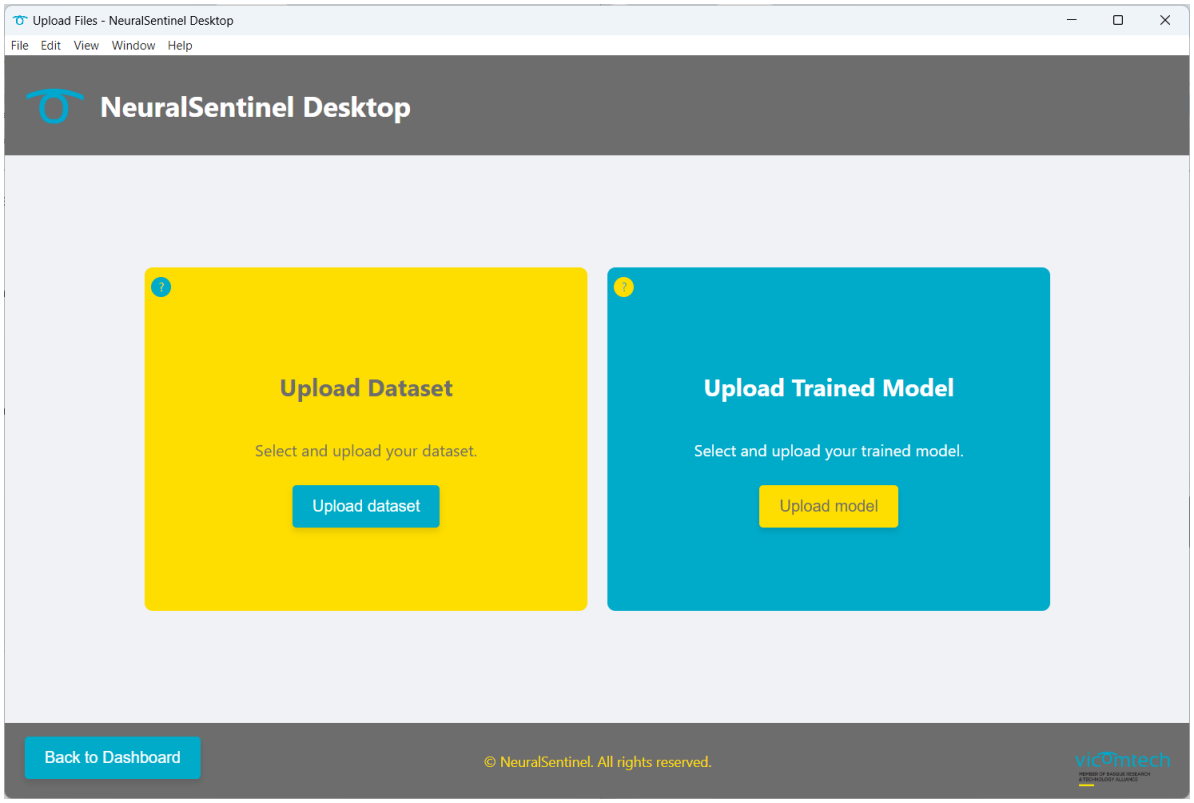
4.3.2 Welcome screen

The welcome (index) screen serves as the introduction to NeuralSentinel Desktop. It provides users with a welcome message explaining the app's purpose, which helps them quickly understand its functionality. This screen includes two main buttons: "Upload files" to go to the upload page for datasets or models, and "View dashboard" to access the dashboard where users can see visualizations, metrics, and manage their files. The screen adheres to consistent branding with the top banner and the Vicomtech logo at the bottom, providing a cohesive experience. The clean design with clear calls to action ensures easy navigation and helps users move forward seamlessly.



4.3.3 Upload screen

The upload screen allows users to upload datasets and trained models for processing within NeuralSentinel Desktop. The "Upload Dataset" and "Upload Model" buttons facilitate file selection. Question mark tooltips in each section inform users of the recommended file formats. There is also a "Back to Dashboard" button at the bottom, enabling users to return to the dashboard at any time. Like other screens, the upload screen maintains consistent branding with Vicomtech's logos and colors. The design is minimalistic and intuitive, providing clear instructions to guide users through the upload process.



4.3.4 Dashboard / metrics screen

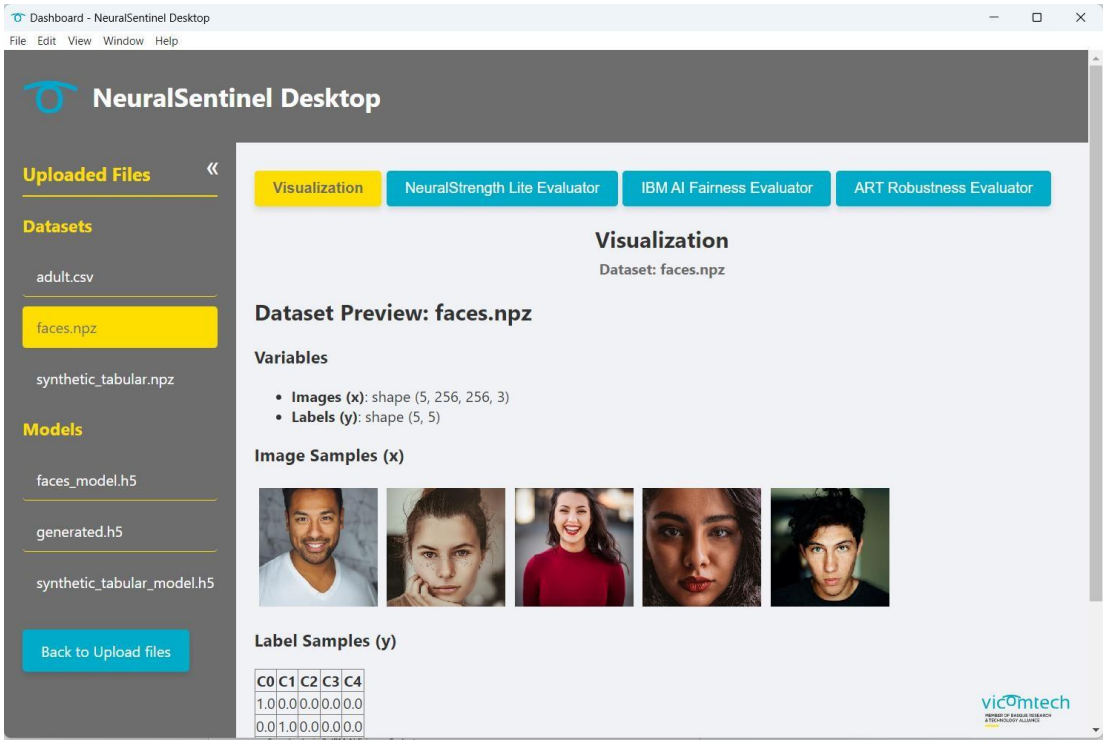
The dashboard screen is the core of the application, where users can access visualizations, manage uploaded files, and view metrics related to their datasets and models. The screen includes a collapsible side navigation bar on the left that allows users to easily switch between sections, selecting either a dataset or a model. A "Back to Upload files" button at the bottom of the sidebar allows users to quickly return to the upload screen to add new datasets or models.

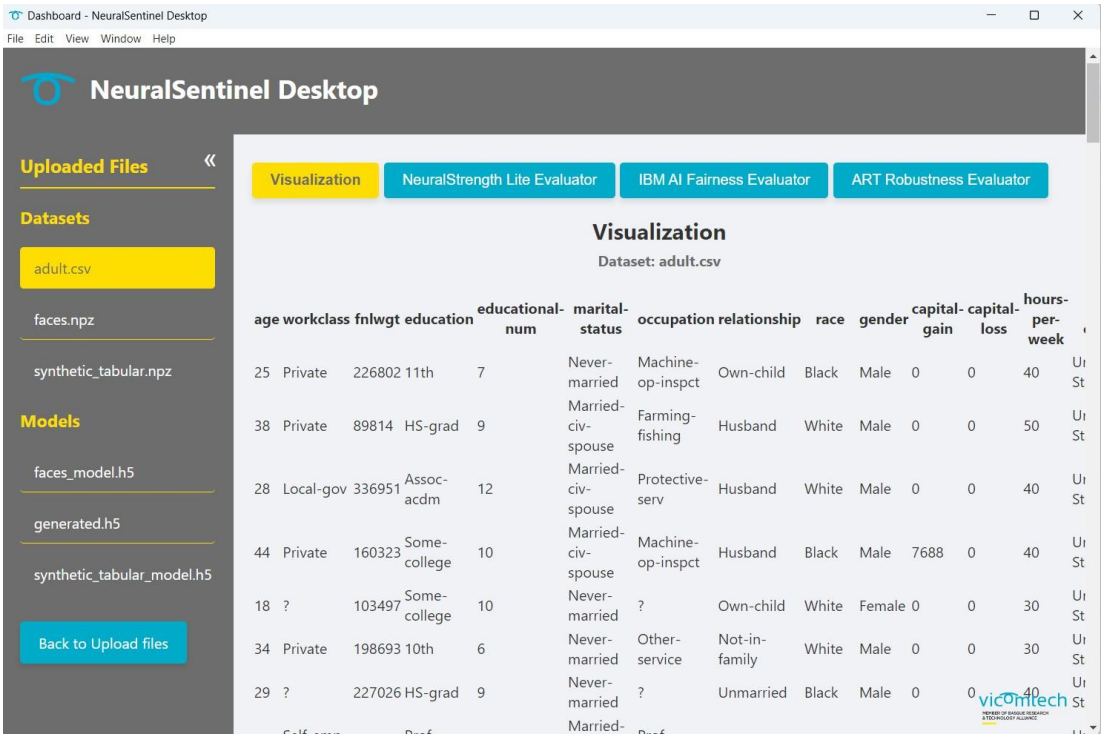
On the right, we find the metrics section. At the top, there is a persistent "Visualization" tab that provides an overview of the selected dataset or model.

Additional functionality is unlocked through plugins, which are integrated as tabs in the main content area. These plugins allow for advanced features such as fairness analysis, performance metrics, and custom visualizations.

4.3.4.1 Visualization tab

The visualization tab is always present on the dashboard and serves as the default view. This section provides a comprehensive view of the selected dataset or model. If a dataset is selected, users can explore tabular data or graphical representations of the dataset's features. If a model is selected, users can see an overview of the model's structure, and other relevant details. The intuitive layout ensures that users can easily navigate between different datasets and models to visualize them and gain insights into their data.


Dashboard - NeuralSentinel Desktop
File Edit View Window Help
NeuralSentinel Desktop
Uploaded Files
Datasets
adult.csv
faces.npz
synthetic_tabular.npz
Models
faces_model.h5
generated.h5
synthetic_tabular_model.h5
Back to Upload files
Visualization
NeuralStrength Lite Evaluator
IBM AI Fairness Evaluator
ART Robustness Evaluator
Visualization
Dataset: faces.npz
Dataset Preview: faces.npz
Variables
Images (x): shape (5, 256, 256, 3)
Labels (y): shape (5, 5)
Image Samples (x)
Label Samples (y)
C0 C1 C2 C3 C4
1.0 0.0 0.0 0.0 0.0
0.0 1.0 0.0 0.0 0.0
vicomtech


Dashboard - NeuralSentinel Desktop
File Edit View Window Help
NeuralSentinel Desktop
Uploaded Files
Datasets
adult.csv
faces.npz
synthetic_tabular.npz
Models
faces_model.h5
generated.h5
synthetic_tabular_model.h5
Back to Upload files
Visualization
NeuralStrength Lite Evaluator
IBM AI Fairness Evaluator
ART Robustness Evaluator
Visualization
Dataset: adult.csv
age workclass fnlwgt education educational-num marital-status occupation relationship race gender capital-gain capital-loss hours-per-week
25 Private 226802 11th 7 Never-married Machine-op-inspct Own-child Black Male 0 0 40
38 Private 89814 HS-grad 9 Married-civ-spouse Farming-fishing Husband White Male 0 0 50
28 Local-gov 336951 Assoc-acdm 12 Married-civ-spouse Protective-serv Husband White Male 0 0 40
44 Private 160323 Some-college 10 Married-civ-spouse Machine-op-inspct Husband Black Male 7688 0 40
18 ? 103497 Some-college 10 Never-married ? Own-child White Female 0 0 30
34 Private 198693 10th 6 Never-married Other-service Not-in-family White Male 0 0 30
29 ? 227026 HS-grad 9 Never-married ? Unmarried Black Male 0 0 40
vicomtech

4.3.4.2 Plugin tabs

Plugins are dynamically added to the dashboard as separate tabs, each offering specialized functionality according to the user's preferences and needs. These plugins are fully integrated with the dashboard, each adding a new tab, allowing users to access additional features without leaving the main interface.

4.3.4.2.1 Sample plugin 1: NeuralStrength Lite Evaluator

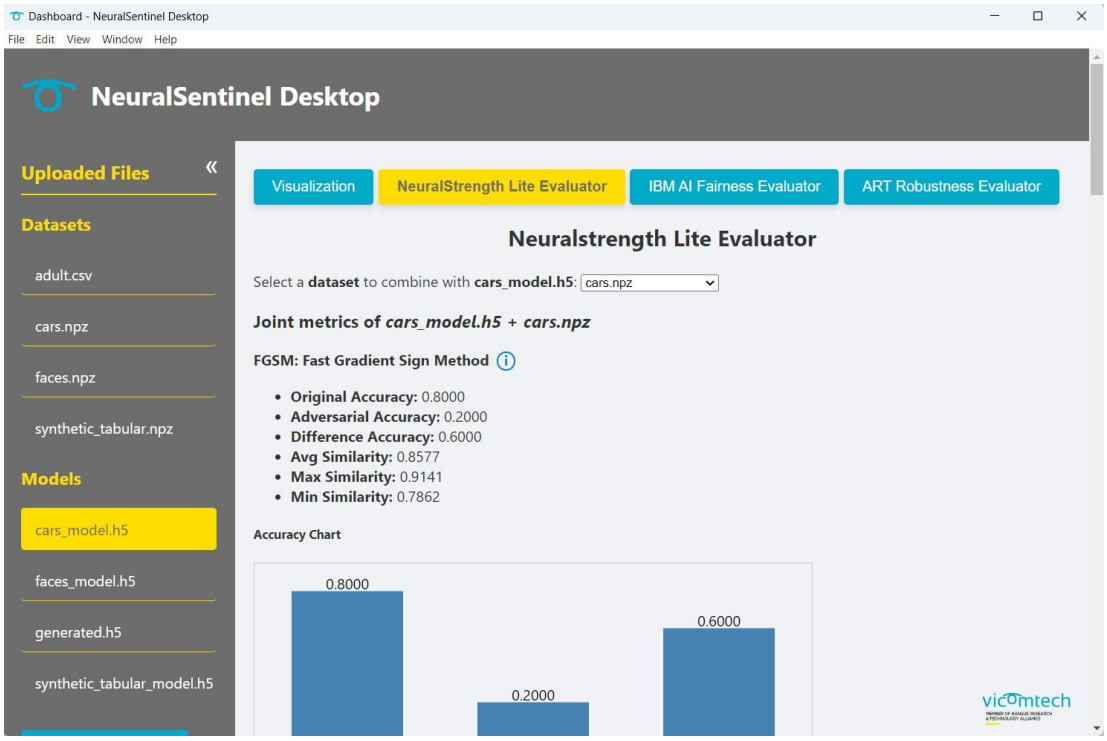
The NeuralStrength Lite Evaluator plugin is useful for verifying that models are sufficiently robust to handle adversarial situations, ensuring that the models are not only effective but also reliable in the face of external manipulations.

This plugin is an early-stage sample (Lite) tool that leverages Vicomtech's proprietary NeuralStrength Lite library, to assess the adversarial robustness of machine-learning models. It automatically normalizes any 1D (tabular) data to the [0, 1] range, subsamples up to 32 examples, and then subjects the model to three attack engines: FGSM, PGD, and BIM, extracting both performance and impact metrics.

It calculates, for each attack:

- **original_accuracy:** accuracy on unperturbed (clean) examples.
- **adversarial_accuracy:** accuracy on adversarially perturbed inputs.
- **difference_accuracy:** absolute drop in accuracy (|original_accuracy − adversarial_accuracy|).
- **avg_similarity:** average similarity score between original and adversarial examples (when provided by the engine).
- **max_similarity** and **min_similarity:** maximum and minimum similarity scores across the sample.
- **impact** (truncated to input_0): per-feature or per-neuron impact values showing how much
- each input dimension was perturbed (returned as a small heatmapable array).

Together, these metrics offer a comprehensive view of a model's resilience, highlighting not only how its predictive accuracy degrades under attack but also how "close" the adversarial examples remain to the originals and which features incur the greatest perturbations.

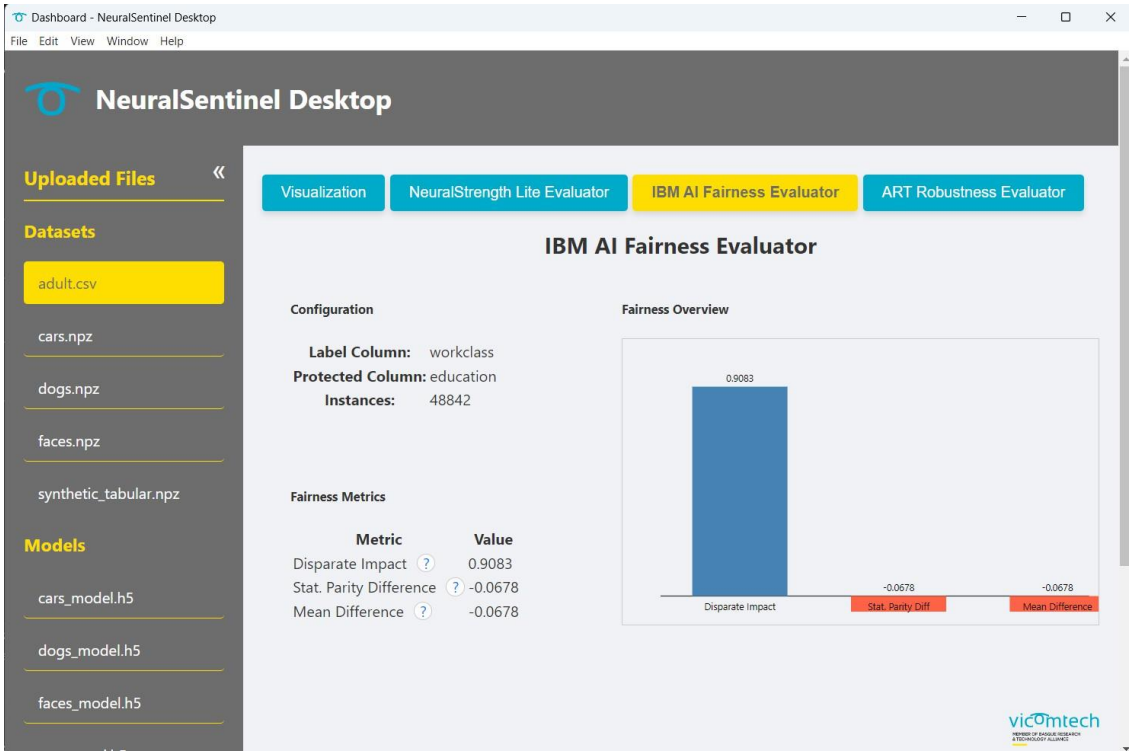


NeuralStrength Lite plugin tab

4.3.4.2.2 Sample plugin 2: IBM AI Fairness Evaluator

The IBM AI Fairness Evaluator plugin is an early-stage sample tool that leverages IBM's AIF360 toolkit to compute a concise set of group-fairness metrics for binary-outcome models. It asks for user input about the label and the protected attribute, automatically binarizes both, wraps the data in an AIF360 BinaryLabelDataset, and then evaluates:

- **disparate_impact**: the ratio of the rate of favorable outcomes for the unprivileged group to that for the privileged group (ideal close to 1.0).
- **statistical_parity_difference:** the difference in favorable-outcome rates between unprivileged and privileged groups (ideal close to 0.0).
- **mean_difference:** the difference in mean predicted labels between unprivileged and privileged groups (another measure of selection-rate gap, ideal near 0.0).



IBM AI Fairness Evaluator plugin tab

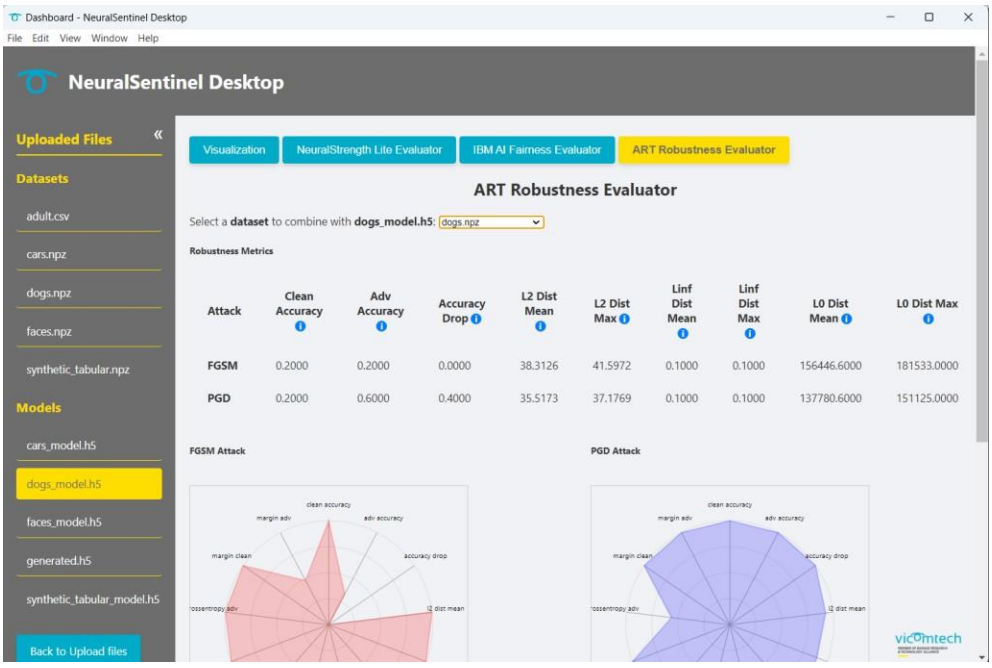
4.3.4.2.3 Sample plugin 3: ART Robustness Evaluator

The ART Robustness Evaluator plugin is an early-stage sample tool that leverages the Adversarial Robustness Toolbox (ART) to compute a comprehensive set of robustness and performance metrics for machine learning models under adversarial attack. It wraps a TensorFlow V2 model in ART's TensorFlowV2Classifier, then evaluates two attack methods (FGSM and PGD) on up to 20 normalized examples, extracting both predictive and perturbation statistics.

It calculates, for each attack (FGSM and PGD):

- **clean_accuracy**: accuracy on unperturbed (clean) examples.
- **adv_accuracy**: accuracy under adversarial perturbations.
- **accuracy_drop**: absolute drop in accuracy.
- **crossentropy_clean**: mean categorical cross-entropy loss on clean data.
- **crossentropy_adv**: mean categorical cross-entropy loss on adversarial data.
- **margin_clean**: mean margin (probability of the true class minus that of the runner-up) on clean examples.
- **margin_adv**: mean margin under adversarial attack.
- **l2_dist_mean** and **l2_dist_max**: mean and maximum L_2 norm of the perturbations.
- **linf_dist_mean** and **linf_dist_max**: mean and maximum $L\infty$ (max-norm) of the perturbations.
- **l0_dist_mean** and **l0_dist_max:** mean and maximum L_0 "norm" (number of features changed) of the perturbations.

Together, these metrics offer detailed insights into how model accuracy, confidence margins, and loss change under attack, as well as into the size and sparsity of the adversarial perturbations. This plugin serves as a proof of concept and could be extended in future releases to include further attack algorithms, support for additional model types, and richer statistical summaries.



ART Robustness Evaluator plugin tab

These plugins add valuable functionality to NeuralSentinel Desktop, covering model robustness, fairness, and performance. While they currently offer essential features, they are also designed as proof of concepts with room for further development and enhancement.

4.4 CONTINUOUS SUPERVISION MODULE

To ensure privacy, proper use, and integrity of the application, a continuous supervision module has been implemented using behavioral biometrics. This module continuously verifies the user's identity throughout their session, detecting unauthorized access or suspicious activity that could compromise data or analysis integrity. Its components and operation are outlined below:

1. Behavioral signal collection
   - A background process (data_collector.py) unobtrusively captures keyboard events (press durations and intervals) and mouse activity (movement speed, click counts, movement-to-click time), as well as a heatmap of interaction zones.
   - Every sampling interval of 5 seconds it produces a CSV file in the user's dedicated directory, ensuring data isolation and full traceability.

2. Authentication model training
   - Upon login, the application aggregates historical behavior data for that user from their folder of CSV files.
   - An Isolation Forest model, preceded by a StandardScaler, is trained on these "legitimate" patterns to characterize normal usage.
   - Training is invoked on demand, allowing periodic retraining or recalibration to adapt to evolving user behavior.

3. Real-time continuous authentication
   - A persistent service (continuous_authenticator.py) reads the newest CSV record at fixed intervals and extracts its feature vector.
   - Each sample is scored by the scaling + Isolation Forest pipeline. If flagged as anomalous, an immediate warning dialog is shown in the UI and the event is logged.
   - The polling interval is configurable (by default every 5 s), balancing detection speed against resource usage.
   - Every anomaly detection is recorded in the application log, with timestamps for future auditing.

4. Privacy and compliance
   - The design follows "Privacy by Design" principles: behavioral data are used solely for authentication and never transmitted externally.

5. Future extensibility
   - This is an initial version of the model, created to validate its core functionality. Future iterations could focus on refining its accuracy, expanding the dataset, and enhancing overall performance.

By integrating this continuous supervision module, NeuralSentinel ensures only legitimate behavior, providing real-time anomaly detection, and creating an audit trail for any security incident, thereby strengthening both application security and regulatory compliance.

4.5 IMPLEMENTATION CONSIDERATIONS

4.5.1 Electron architecture

Before diving into our implementation, it is essential to understand how Electron structures a desktop application under the hood. Electron separates concerns into two isolated runtime contexts: Main and Renderer, in this case with an optional Preload layer that was added as a controlled bridge. This design lets us safely combine rich web-style UIs with low-level system access.

The two primary runtime contexts Electron splits desktop applications into are:

- Main process

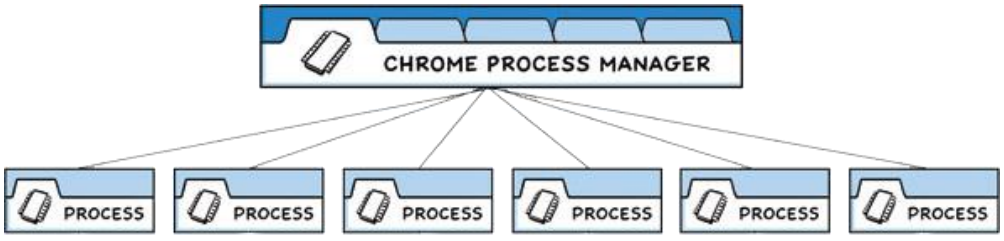
  The Main process is the application's brain. It boots first, creates and manages BrowserWindow instances, and handles all privileged operations: reading/writing files, talking to the SQLite user database, spawning the embedded Python runtime, showing native dialogs, and orchestrating auto-updates. Because it runs with full Node.js and OS privileges, we centralize every disk- or network-bound task here. All Renderer-to-system interactions flow through IPC handlers registered in this process.

- Renderer process(es)

  Each BrowserWindow hosts its own Renderer process, essentially a Chromium tab isolated from Node.js by default. Renderers are responsible solely for presenting HTML/CSS views and running the UI-side JavaScript controllers (login.js, upload.js, dashboard.js). They fire user events (clicks, form submissions, tab changes), but cannot touch the file system or spawn subprocesses directly.

- Preload script

  To bridge the gap safely, we load a small Preload script into every Renderer before our UI code runs. Using contextBridge.exposeInMainWorld(), it injects a minimal window.electronAPI object exposing only the exact IPC methods the UI needs (e.g. getUploadedFiles(), uploadAndCalc(), computeComboMetrics(), log(), getPluginMeta(), getPluginTabHtml(), etc.). This "allowlist" approach hardens the app by preventing untrusted web code from calling arbitrary Node or OS APIs.



Electron's Process Model [3]

[3] Source: https://www.electronjs.org/docs/latest/tutorial/process-model

In the NeuralSentinel Desktop application, the Electron main process and the renderer process (where the user interface operates) must communicate with the Python backend to handle various tasks such as data processing and plugin interactions. The data flow between these processes is handled through Inter-Process Communication (IPC), allowing seamless interaction while maintaining performance. Here is how the communication works:

- Electron main → Python

  When the user interacts with the front-end, Electron's Main process spawns a Python process (via child_process.spawn under the hood). For example, when a file is uploaded or a plugin is selected, the Main process sends relevant information (file paths, plugin identifiers, configuration flags) as command-line arguments to the Python executable. This ensures that heavy processing (metrics computation, model evaluation, plugin logic) occurs in the backend, keeping the UI thread unblocked.

- Python → Electron

  After the Python backend completes its task (e.g., calculating metrics, running a plugin), it returns results as JSON streamed over stdout. The Electron Main process captures this output, parses the JSON, and then **forwards it** to the appropriate Renderer via IPC (e.g., using ipcMain.handle and ipcRenderer.invoke). The Renderer process then updates the UI in real time, displaying computed metrics or plugin visualizations.

By utilizing IPC, the application's heavy data computations are isolated from the UI process. This model ensures that the front-end remains responsive, with no freezes or lags, even when dealing with complex data processing tasks. The UI is updated asynchronously as results arrive, providing a fluid, uninterrupted user experience.

Organizing our app this way, we achieve two critical goals:

- **Security**: Untrusted UI code lives in a locked-down Chromium sandbox, only vetted IPC channels cross into privileged territory.
- **Maintainability:** We cleanly separate presentation (Renderer), business logic and I/O (Main), and the controlled bridge (Preload), mirroring a service-oriented microkernel.

4.5.2 MVC alignment

As mentioned before, in NeuralSentinel, we embrace a "light" MVC-inspired structure by distributing responsibilities across Electron's multi-process architecture and our Python backend:

1. Model

The Model lives in the Electron main process and the Python backend. All persistence (SQLite reads/writes), file I/O, and security-evaluation algorithms execute here. When the user uploads a file or requests an analysis, the Main process spawns an isolated Python subprocess, hands over file paths and parameters, and receives JSON-formatted results. This

ensures that every privileged operation and core computation remains centralized, testable, and decoupled from presentation concerns.

2. View

Views run entirely in the Renderer processes (Chromium tabs) and consist of HTML/CSS templates for the application's GUI: login page, upload form, and dashboard panels (file lists, charts, plugin tabs). They display the Model's state without any direct knowledge of how data is stored or how logic runs, simply emitting user events (clicks, form submissions, tab changes) to the Controller layer.

3. Controller

Controller logic resides in two places: the Preload script and the Renderer controllers (login.js, upload.js, dashboard.js). The Preload layer, added for security rather than by Electron's default, exposes a minimal window.electronAPI with only the IPC methods the UI needs (e.g. getUploadedFiles(), uploadAndCalc(), computeComboMetrics(), log(), getPluginMeta()). Renderer controllers then catch user interactions, call these IPC methods, and update the DOM with the responses. This strict data-flow: UI → Controller → IPC → Main (Model) → back to Controller → View, preserves separation of concerns, keeps untrusted code sandboxed, and maintains high testability and flexibility without imposing rigid class hierarchies.

By distributing these roles across processes rather than within monolithic classes, we retain the separation, testability, and flexibility of MVC, while leveraging Electron's process isolation and our Python-based plugin ecosystem.

4.5.3 Plugin architecture application

NeuralSentinel's plugin framework resides entirely in its Python backend and realizes the plugin pattern with application-specific contracts and workflows:

1. Configuration

A central plugin_config.yaml lists every plugin and its exposed functions, allowing administrators to enable or disable modules and entry points without touching code.

2. Discovery and registration

On startup, the Electron Main process invokes a Python launcher (run_plugins.py). This launcher reads plugin_config.yaml, dynamically imports each enabled package via Python's importlib, and examines its manifest (either plugin.yaml or embedded metadata) to register two required entry points per plugin:

- **Evaluator**: a function that consumes model and dataset paths to compute security metrics.
- **Visualizer**: a function that formats those metrics into HTML snippets or JSON structures.

3. Execution hooks

When combined metrics are requested (e.g. via the Dashboard), the Main process bundles file locations and calls the Python runner. The runner sequentially:

- Invokes each plugin's Evaluator, collecting raw results.
- Passes those results into the corresponding Visualizer, aggregating the returned HTML or JSON.

4. Renderer integration

In the Renderer's dashboard controller (dashboard.js), a tab button is generated for each registered plugin name.

Clicking a tab fires getTabHtml(pluginName), triggering an IPC round-trip: Renderer → Main → Python → Main → Renderer.

The returned fragment is injected into the plugin tab's content container, seamlessly extending the View without any Renderer-side knowledge of plugin internals.

5. Failure isolation

Every call into plugin code is wrapped in a try/catch. If an Evaluator or Visualizer raises an exception, the error is logged (via the shared logging service) and returned as a user-friendly message, and other plugins continue to run unaffected.

By binding these steps together, NeuralSentinel cleanly extends its core Model layer with third-party evaluation and visualization modules, while keeping the Controller and View layers entirely agnostic to plugin implementation details. New security evaluators or custom charts can be added at runtime by simply dropping a compliant Python package into the plugins directory and toggling it in the YAML, delivering true extensibility without any core modifications.

4.5.4 Data flow

1. Data ingestion
   When the user provides a file path, it is sent from the front-end (Electron) to the Python backend. The Python backend then processes the file, determining the appropriate format (e.g., dataset or model) and loading the data accordingly.

2. Preprocessing
   Some plugins require pre-processing steps before calculations can begin. This may involve verifying the file structure, handling missing or invalid data, and standardizing numeric ranges or formats to ensure consistency across the dataset.

3. Metric computation

Each plugin performs its own specific computation logic. For example, a fairness plugin might evaluate how different protected attributes impact outcomes, while a robustness plugin could generate adversarial samples to assess a model's vulnerability to attacks.

4. Results aggregation
   The findings from the plugin are gathered and converted into a consistent format (e.g., a JSON object). This standardized output ensures that the results can be easily processed and displayed in the user interface.

5. Display and storage
   The JSON data is sent back to the Electron front-end, where the UI is updated in real time to reflect the computed metrics. Additionally, the results may be logged for future retrieval, export, or further analysis, providing a record of the computations performed.

4.5.5 Testing

A comprehensive suite of automated tests has been developed to validate core application logic, ensure correct integration with external components, and guard against regressions. Key testing areas include:

- Utility functions and data sanitization

  Independent helper routines, such as HTML escaping and path resolution, are exercised under both development and packaged scenarios, with external dependencies mocked to isolate pure logic.

- Persistence and authentication

  Database-access routines are verified against a controlled SQLite fixture to confirm correct retrieval of user records, proper password hashing behavior, and graceful handling of missing entries.

- Python backend integration

  Invocation of the embedded Python runtime is simulated to check executable resolution, script availability checks, and robust error handling for missing files or non-zero exits. Successful execution paths are validated by parsing expected JSON output.

- Caching and performance optimization

  The in-memory metrics cache is exercised to ensure that repeated requests for identical inputs bypass unnecessary backend calls, while cache invalidation semantics preserve correctness when inputs change.

- Plugin resolution and extension points

  The plugin discovery and registration workflow is tested in isolation, confirming that extension points are correctly identified, callbacks are registered, and failures within individual plugins are contained without impacting overall application stability.

- Electron process logic

  Cross-process IPC handlers and preload-exposed APIs are validated under mocked environments to ensure that renderer-to-main communication conforms to security constraints, and that all privileged operations remain safely confined to the main process.

By mocking external systems (file I/O, database paths, subprocess launches) and exercising each layer independently, these tests deliver fast, deterministic validation of critical pathways and maintain an overall line-coverage comfortably above industry benchmarks.

4.5.6 Brand identity

In the development of the application, significant attention has been given to aligning the visual design and user interface with Vicomtech's brand identity. This ensures that the application not only functions effectively but also reflects the organization's values and reinforces its professional presence. The integration and consistency of brand elements like colors, logos, and icons has been carefully implemented to create a seamless user experience that is both recognizable and cohesive.

1. Corporate colors

   The application uses the official Vicomtech color palette to create a visually consistent experience. The primary colors include:

   - Primary blue (#00ABC9)
   - Complementary gray (#6D6D6D)
   - Accent yellow (#FEDD00)

These colors are used in the sidebar, primary buttons, banners and various elements to ensure a cohesive look and feel that reflects the Vicomtech's established identity.

Vicomtech's corporate colors [4]

[4] Source: https://www.vicomtech.org/en/vicomtech/branding

2. Logo:

Vicomtech's official logo is placed in the bottom-right corner of all screens. This choice reinforces the brand's presence at all times without interfering with the main interface.

Vicomtech's logo [4]

3. Application icon

The application icon has been configured to appear both on the taskbar and at the top-left of the window, alongside the application's title in the top banner. This ensures instant brand recognition while unifying the look of the software with the company's other products.

Vicomtech's icon [4]

By consistently integrating these resources (colors, logo, and icon), the application is visually integrated into Vicomtech's ecosystem, promoting a unified and professional user experience while reinforcing brand credibility and fostering user trust.

4.5.7 Data visualization

The app's design prioritizes legibility, consistency, and intuitiveness to provide an optimal user experience, especially during extended use.

1. Legibility:

High-contrast color schemes are used, with white on blue and black on light gray for buttons and text, ensuring clarity and reducing strain.

A gray background is applied to larger content areas to minimize visual fatigue during prolonged use, avoiding the harshness of a white background.

2. Consistency:

The application maintains a consistent color palette aligned with Vicomtech's brand, using blue for primary actions, yellow for highlights, and gray for neutral areas, creating a cohesive and professional design.

UI elements and buttons follow uniform styles for easy navigation, making actions predictable.

3. Intuitiveness:

Interactive charts and graphs present security metrics in a clear and digestible way, with tooltips providing additional context when needed.

The interface is divided into clearly separated tabs, such as Visualization and Plugin tabs, to distinctly separate functionalities, helping users navigate between tasks without confusion.

In summary, the data visualization design in NeuralSentinel ensures that users can easily interpret security metrics with minimal effort, while also providing a comfortable viewing experience over extended periods.

DEFINITIONS, ACRONYMS, ABBREVIATIONS

1. MVC - Model-View-Controller (software architecture pattern)
2. EU AI Act - European Union Artificial Intelligence Act
3. AI - Artificial Intelligence
4. ML - Machine Learning
5. DL - Deep Learning
6. MPC - Secure Multi-Party Computation
7. OECD - Organisation for Economic Co-operation and Development
8. NIST - National Institute of Standards and Technology
9. AI RMF - AI Risk Management Framework (by NIST)
10. PPML - Privacy-Preserving Machine Learning
11. ROI - Return on Investment
12. NIS2 - Network and Information Systems Directive (EU)
13. DORA - Digital Operational Resilience Act (EU)
14. GDPR - General Data Protection Regulation (EU)
15. ART - Adversarial Robustness Toolbox (by IBM)
16. CLI - Command-Line Interface
17. FGSM - Fast Gradient Sign Method (adversarial attack)
18. GUI - Graphical User Interface
19. AIF360 - IBM's AI Fairness 360
20. AUC - Area Under the Curve (metric for model performance)
21. API - Application Programming Interface
22. SaaS - Software as a Service

23. GRC - Governance, Risk, and Compliance
24. GCP - Google Cloud Platform
25. DP - Differential Privacy
26. SEAL - Simple Encrypted Arithmetic Library (Microsoft)
27. CSV - Comma-Separated Values
28. CRISP-DM - Cross-Industry Standard Process for Data Mining
29. IPC - Inter-process communication
30. REST API - REpresentational State Transfer Application Programming Interface
31. PMI - Project Management Institute
32. HDF5 (.h5) - Hierarchical Data Format version 5
33. ONNX - Open Neural Network Exchange (model format)
34. HTTP - Hypertext Transfer Protocol
35. CSP - Content Security Policy
36. XSS - Cross-Site Scripting
37. JSON - JavaScript Object Notation

APPENDICES

Appendix 1: Developer's guide to building NeuralSentinel plugins

Welcome to the NeuralSentinel plugin development guide. NeuralSentinel is a cross-platform desktop application designed to deliver robust, intuitive security evaluations of AI models and datasets. In an era of increasing adversarial threats, privacy concerns, and evolving regulations, it's critical to adapt and extend assessment capabilities to meet new challenges.

To support this, NeuralSentinel employs a modular plugin architecture that isolates custom code and ensures forward compatibility. Plugins let you seamlessly integrate custom analyses, such as adversarial robustness tests, bias audits, or privacy checks, into the core system, without modifying existing code or disrupting the user experience.

Below is a high-level overview of the guide before diving into detailed examples and code snippets. This guide walks you through:

1. **Prerequisites**
2. **Plugin architecture overview**
3. **Directory layout and configuration**
4. **Writing an Evaluator**
5. **Writing a Visualizer**
6. **Integration and runtime flow**
7. **Error handling and isolation**
8. **Best practices**
9. **Example plugin**

1. Prerequisites

Before you begin, make sure you have:

- **Python 3.9** installed.
- Basic familiarity with Python modules and classes.
- Access to the NeuralSentinel codebase with the python_backend/ directory.
- An IDE or text editor configured for Python development.

2. Plugin architecture overview

NeuralSentinel's plugin system is built around three layers:

- **Core application (Electron)** handles the GUI, IPC messaging, and plugin orchestration.
- **Python backend** executes plugin logic in isolated subprocesses.
- **Plugin components** consist of:
    - **Evaluator**: computes metrics from a model and dataset
    - **Visualizer**: renders metrics as HTML for display

Each layer communicates via well-defined IPC channels to maintain security boundaries.

At startup, the application reads a central YAML configuration file, discovers enabled plugins, and dynamically loads their evaluator classes. When the user requests analysis, the backend runs each evaluator method listed in the config and caches results. The frontend then invokes visualizers on demand, passing cached data to render HTML tabs.

3. Directory layout and configuration

3.1 File structure

Place your plugin under:

```python
python_backend/
└── plugins/
    └── your_plugin_name/
        ├── __init__.py
        ├── evaluator.py
        └── visualizer.py
```

- __init__.py: an empty file to mark the folder as a Python package.
- evaluator.py: defines the Evaluator class.
- visualizer.py: defines the Visualizer class.

3.2 plugin_config.yaml

The root configuration file lives at python_backend/plugin_config.yaml. It should follow this schema, respecting proper indentation:

```python
plugins:
  - name: "neuralstrength_lite_evaluator"
    display_name: "NeuralStrength Lite Evaluator"
    enabled: true
    functions:
      - "evaluate_robustness"

  - name: "ibm_ai_fairness_evaluator"
    display_name: "IBM AI Fairness Evaluator"
    enabled: true
    functions:
      - "compute_fairness"

  - name: "your_plugin_name"
    display_name: "Your Tab Name For The Plugin"
    enabled: true
    functions:
      - "evaluate_robustness"
```

- **name**: unique plugin identifier (matches folder name).
- **display_name**: human-readable tab label.
- **enabled**: toggle plugin on/off without removing code.
- **functions**: list of method names implemented by your Evaluator.

The system assumes:

- **Evaluator Module**: plugins.<name>.evaluator, containing a class named Evaluator.
- **Visualizer Module**: plugins.<name>.visualizer, containing a class named Visualizer.

4. Writing an Evaluator

The Evaluator computes one or more metrics, exposed as methods listed in functions. It must subclass the BaseEvaluator interface.

4.1 BaseEvaluator contract

```python
from core.interfaces import BaseEvaluator


class Evaluator(BaseEvaluator):
    def __init__(self, logger):
        self.logger = logger
        self.meta = {
            "name": "your_plugin_name",       # must match
config name
            "display_name": "Your Plugin Title",
            "functions": ["evaluate_robustness"]  # must
match config functions
        }


    def evaluate_robustness(self, model_path: str,
dataset_path: str) -> dict:
        """
        Example metric method.
        Args:
          model_path: path to the .h5 model file
          dataset_path: path to the .npy dataset file
        Returns:
          {"robustness_score": 0.95}
        """
```

```
        try:
            # 1. Load model and dataset
            # 2. Compute robustness metric...
            score = 0.95
            return {"robustness_score": score}
        except Exception as e:
            self.logger.error(f"[{self.meta['name']}]
{e}")
            return {"error": str(e)}
```

4.2 Key points

- **Class name and location**: Evaluator in plugins/<name>/evaluator.py.
- **Meta**: must include name, display_name, and functions exactly matching the YAML.
- **Method signatures**: each function takes (model_path, dataset_path) and returns a JSON-serializable dict.
- **Error handling**: catch exceptions, log, and return {"error": msg}.

5. Writing a Visualizer

The Visualizer turns raw metrics into rich HTML for the dashboard tab.

5.1 BaseVisualizer contract

```python
class Visualizer(BaseVisualizer):
    def __init__(self):
        self.tab_id = "your_plugin_name"  # matches plugin name
        self.name   = "Your Plugin Title"  # matches display_name

    def get_tab_metadata(self) -> dict:
        return {"id": self.tab_id, "name": self.name}

    def render_content(self, payload: dict) -> str:
        """
        Args:
          payload: full map of {plugin_name: {function: result}}
        Returns:
          HTML fragment
        """
```

```
        data = payload.get(self.tab_id, {})
        html = [f"<h2>{self.name}</h2>"]

        if 'error' in data:
            html.append(f"<div class='error'>Error:
{data['error']}</div>")
        else:
            html.append('<ul>')
            for func, result in data.items():
                html.append(f"<li><strong>{func}:</strong>
{result}</li>")
            html.append('</ul>')

        return '\n'.join(html)

if __name__ == '__main__':
    payload_json = sys.argv[1]
    payload = json.loads(payload_json)
    viz = Visualizer()
    print(viz.render_content(payload))
```

5.2 Key points

- **Tab metadata**: get_tab_metadata() must match name and display_name.
- **render_content()**: receives a nested dict: {plugin_name: {func_name: value}}.
- **Standalone**: script invoked with the full JSON payload.

6. Integration and runtime flow

1. App startup

run_plugins.py parses plugin_config.yaml, imports each Evaluator, and records metadata.

2. User uploads files

When files are uploaded, the backend:

- iterates enabled plugins
- calls each configured function: Evaluator().<func>(model_path, dataset_path)
- aggregates results into {plugin_name: {...}}
- caches per-session.

3. UI Tab Generation

- The renderer requests plugin metadata, builds tabs using display_name.

4. Tab rendering

- On click, the renderer calls get-tab-html, passing cached plugin results. The main process spawns visualizer.py.

5. Display
   - HTML from the Visualizer is injected into the dashboard.

7. Error handling and isolation

Make sure you:

- Surround plugin import and function calls with try/except to prevent crashes.
- Log all exceptions via the provided logger.
- Return {"error": message} to allow the UI to show errors gracefully.

8. Best practices

Developing high-quality plugins requires more than just working code. Following these best practices will help ensure your extensions are reliable, maintainable, and performant, both for yourself and for other users who may adopt your work.

- Adhere to consistent naming conventions

  Ensure your plugin folder name, the name field in plugin_config.yaml, and the tab_id in your Visualizer all match exactly in snake_case. This alignment prevents import errors and simplifies troubleshooting when you or others revisit the codebase.

- Minimize external dependencies

  Only include libraries that are absolutely necessary for your plugin's functionality. Keeping dependencies lightweight reduces installation issues, speeds up loading times, and lowers the risk of version conflicts with the core application or other plugins.

- Provide clear, user-friendly descriptions

  Use the display_name field and your meta["display_name"] in the Evaluator to give end users an immediate understanding of what your plugin does. Brief but descriptive labels in the UI help users discover and trust your functionality.

- Implement efficient caching strategies

  If your metric computations are resource-intensive, cache results for a given (model_path, dataset_path) pair. Avoid re-running the same analysis multiple times during a session: this conserves CPU time and improves the responsiveness of the dashboard.

- Handle errors gracefully

  Anticipate potential failures: file-not-found, corrupt data, or unexpected input, and capture them in try/except blocks. Log detailed error information via the provided logger and return user-friendly error messages so the UI can inform users without crashing.

- Document your code and configuration

  Update this guide and your plugin_config.yaml whenever you add or modify functions. Include docstrings for each Evaluator method and inline comments for non-obvious logic. Well-documented plugins are easier to maintain, share, and debug.

9. Example Plugin: "hello_world"

1. **Directory:** plugins/hello_world/
2. **plugin_config.yaml** entry:

```yaml
plugins:
  - name: "hello_world"
    display_name: "Hello World"
    enabled: true
    functions:
      - "say_hello"
```

3. **evaluator.py**:

```python
from core.interfaces import BaseEvaluator


class Evaluator(BaseEvaluator):
    def __init__(self, logger):
        self.logger = logger
        self.meta = {"name": "hello_world",
"display_name": "Hello World", "functions": ["say_hello"]}


    def say_hello(self, model_path, dataset_path):
        return {"message": "Hello, world!"}
```

visualizer.py:

```python
import sys, json
from core.interfaces import BaseVisualizer

class Visualizer(BaseVisualizer):
    def __init__(self):
        self.tab_id = "hello_world"
        self.name   = "Hello World"

    def get_tab_metadata(self):
        return {"id": self.tab_id, "name": self.name}

    def render_content(self, payload):
        data = payload.get(self.tab_id, {})
        html = [f"<h2>{self.name}</h2>"]
        if 'message' in data:
            html.append(f"<p>{data['message']}</p>")
        return '\n'.join(html)

if __name__ == '__main__':
    payload = json.loads(sys.argv[1])
    print(Visualizer().render_content(payload))
```

Appendix 2: User's guide

NeuralSentinel User's Guide

Table of contents

- Introduction
- System requirements
- Installation and launch
- Getting Started
  - Login screen
  - Welcome screen
- Uploading your data
  - Upload dataset
  - Upload model
- Dashboard overview
  - Navigation pane
  - Visualization tab
  - Plugin tabs
- Using the built-in plugins
  - NeuralStrength Lite Evaluator
  - IBM AI Fairness Evaluator
  - ART Robustness Evaluator
- Continuous supervision module

Introduction

Welcome to **NeuralSentinel**, a desktop application designed to help you evaluate the security, robustness, fairness, and privacy of your AI models and datasets. Through an intuitive graphical interface, you can upload your files, run a suite of analyses, and view rich visualizations, all without writing a single line of code.

System requirements

- **Operating System:** Windows 10+
- **Memory:** ≥ 6.60 GB free for installation, plus space for your data
- **CPU/GPU:** Multi-core CPU; GPU optional
- **Dependencies:**
  - Python 3.9 runtime (already bundled)
  - Electron v25 (already bundled)

Installation and launch

1. **Download** the latest installer for NeuralSentinel.
2. **Double-click** on the installer and wait until the download is completed.
3. Once installed, **launch** "NeuralSentinel" from your Windows Start menu, or desktop shortcut.
4. On first launch, the app will verify that its bundled Python and Electron runtimes are valid, this typically takes just a few seconds.

Getting started

Login screen

- Authenticate your user account before accessing the app.
- **Fields:**
  - **Username**
  - **Password**
- **Behavior:**
  - On successful login, the **data collector** and **continuous authenticator** services start automatically in the background.
  - On failure, an inline error message prompts you to retry.

Welcome screen

- Entry point after login, with two main actions:
  - **Upload Files** → go to the Upload screen
  - **View Dashboard** → go directly to the Dashboard (if you've already uploaded files)

Uploading your data

Upload dataset

1. Click **Upload Files** on the Welcome screen.
2. Select **Upload Dataset**.
3. Choose a file (supported formats: .csv, .npy, .npz, .h5).
4. Hover over the question-mark tooltip for guidance on format and size limits.
5. Once uploaded, the dataset appears in your file list.

Upload model

1. On the same screen, select **Upload Model**.
2. Choose a file (supported formats: Keras HDF5 .h5).
3. Confirm the upload; your model is added to the file list.

At any time, click **Back to Dashboard** to proceed once you have at least one dataset or model uploaded.

Dashboard overview

Navigation pane

- Located on the left, it lists all uploaded **Datasets** and **Models**.
- Click an item to select it.
- **Back to Upload** button at the bottom returns you to the Upload screen.

Visualization tab

- Always present as the first tab.
- **For a Dataset:** shows a table preview, basic statistics (mean, std, missing values), and feature histograms.
- **For a Model:** shows architecture summary, layer counts, and weight distribution plots.

Plugin tabs

- Appear dynamically after any built-in or third-party plugin is registered.
- Each plugin adds a tab labeled by its **display name**.
- Click a plugin tab to run its analysis on the selected dataset or model.

Using the built-in plugins

NeuralStrength Lite Evaluator

Assesses adversarial robustness on tabular models.

- **What it does:** Runs FGSM, PGD, and BIM attacks on up to 32 samples.
- **Outputs:** Clean vs. adversarial accuracy, accuracy drop, similarity metrics, per-feature impact heatmap.

IBM AI Fairness Evaluator

Audits binary-outcome models for group fairness.

- **What it does:** Wraps your data in AIF360, computes disparate impact, statistical parity difference, mean difference.
- **Inputs:** You'll specify the **label** column and **protected attribute**.

ART Robustness Evaluator

Leverages IBM's Adversarial Robustness Toolbox for deep models.

- **What it does:** Runs FGSM and PGD on normalized inputs.
- **Outputs:** Clean/adv accuracy, loss, confidence margin changes, and $L_0/L_2/L\infty$ norms of perturbations.

Continuous supervision module

Monitors user behavior to detect anomalous activity.

1. **Data Collector:** Samples keyboard/mouse usage every 5 s.
2. **Authenticator:** Isolation Forest flags deviations; if suspicious, you'll see an on-screen warning.
3. **Privacy:** All behavioral data stays local, used only for session continuity and anomaly alerts.